

Deploying Metabase on AWS Fargate (ECS Service)

by Mohamad Ikhsan Nurulloh



AWS Fargate

Important Disclaimer - Before You Dive In

- **No Endorsement:** This guide is for educational purposes only and does not imply endorsement by Metabase or AWS.
- **Testing Only:** Intended for testing and development, not production environments.
- **Refer to Official Docs:** Always consult official Metabase and AWS documentation for production use.
- **Proceed at Your Own Risk:** Use this guide at your own risk and be aware of potential deployment implications.

AWS Fargate in a Nutshell



AWS Fargate

AWS Fargate is a technology that you can use with Amazon ECS to run containers without having to manage servers or clusters of Amazon EC2 instances. With AWS Fargate, you no longer have to provision, configure, or scale clusters of virtual machines to run containers. This removes the need to choose server types, decide when to scale your clusters, or optimize cluster packing.

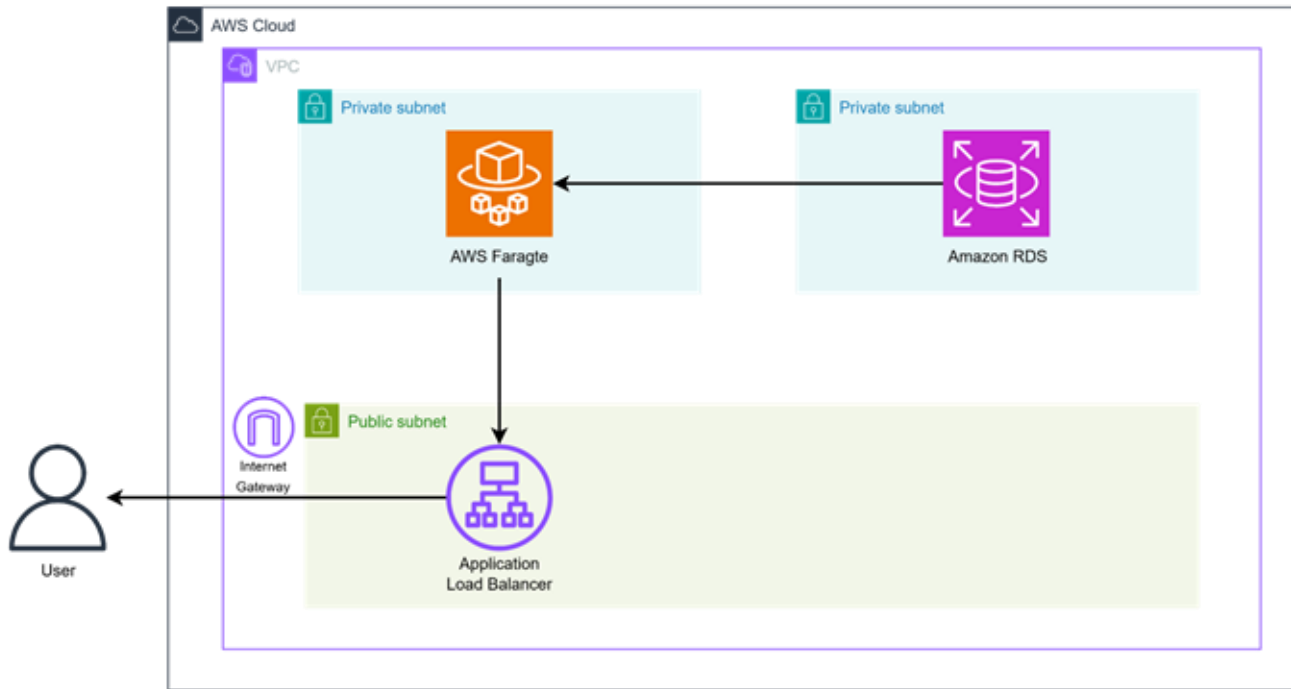
Why Fargate for Metabase?



AWS Fargate

Deploying Metabase on Amazon ECS Fargate is a simple and low-cost way to run containers without managing servers. It removes the need for EC2 instances and load balancers, scales easily and reduces operational work, ideal for small to medium deployments or development environments.

High-Level Architecture

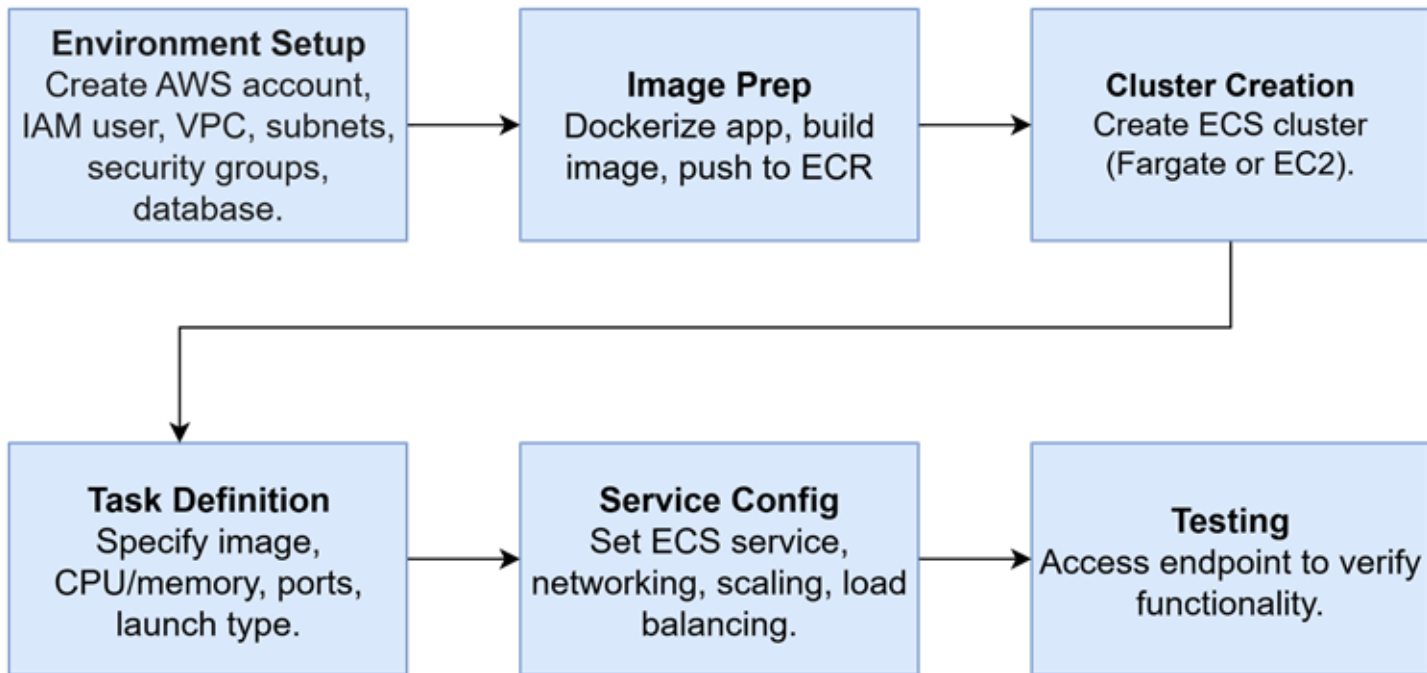


Overall Architecture: Metabase runs on AWS Fargate behind an Application Load Balancer (ALB) in a public subnet, which handles HTTPS traffic from users. The Metabase containers in private subnets store metadata in Amazon RDS. Security groups restrict traffic so only the ALB can reach Fargate and only Fargate can access RDS, delivering a secure, scalable, and fully managed analytics platform.

Inside the Architecture - Key Components Explained

- **VPC with Public & Private Subnets:** Public subnet hosts the ALB; private subnets host Fargate tasks and Amazon RDS for isolation and high availability.
- **Application Load Balancer (ALB):** Handles HTTPS from users via the Internet Gateway and securely routes requests to Fargate tasks.
- **AWS Fargate (ECS Service):** Runs Metabase containers without server management, scaling automatically to match demand. Outbound internet access is not available.
- **Amazon RDS:** Provides a managed PostgreSQL database in private subnets for storing all Metabase metadata and configuration.
- **Security:** Security groups restrict traffic so only ALB → Fargate → RDS communication is allowed, ensuring a highly controlled and secure network flow.

Deployment Roadmap



Building the Foundation: Network Infrastructure Setup

The screenshot displays the AWS Management Console interface for a VPC named 'vpc-0dfb5ebba4ea6d2b4 / Metabase VPC'. The console is in the 'United States (N. Virginia)' region.

Details:

- VPC ID:** vpc-0dfb5ebba4ea6d2b4
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Enabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-090f1a1ed735ca019
- Main network ACL:** acl-0055a1e0136e6ca84
- Default VPC:** No
- Main route table:** rtb-070a993bc5b6628ea / Private Route Table
- IPv6 CIDR (Network border group):** -
- Network Address Usage metrics:** Disabled
- IPv4 CIDR:** 10.0.0.0/16
- Route 53 Resolver DNS Firewall rule groups:** -
- IPv6 pool:** -
- Owner ID:** 797666781058

Resource map:

- VPC:** Your AWS virtual network. Metabase VPC.
- Subnets (3):** Subnets within this VPC.
 - us-east-1a:** Public Subnet, Private Subnet 1.
 - us-east-1b:** Private Subnet 2.
- Route tables (2):** Route network traffic to resources.
 - Public Route Table
 - Private Route Table
- Network Connections (1):** Connections to other networks.
 - Metabase IGW

The resource map shows the VPC connected to three subnets (Public Subnet, Private Subnet 1, and Private Subnet 2). The Public Subnet is connected to the Public Route Table, and the Private Subnets are connected to the Private Route Table. The Public Route Table is connected to the Metabase IGW, which is also connected to the Private Route Table.

A VPC with public and private subnets was created and an Internet Gateway attached. Route tables and security groups for ALB, ECS tasks, and RDS were configured.

Security Group Traffic Control

The screenshot displays the AWS Management Console interface for a Security Group. The breadcrumb navigation shows the path: VPC > Security Groups > sg-0137e41ed35a5be2b - Metabase SG. The page title is 'sg-0137e41ed35a5be2b - Metabase SG'. A 'Details' section provides the following information:

- Security group name:** Metabase SG
- Security group ID:** sg-0137e41ed35a5be2b
- Description:** Metabase SG
- VPC ID:** vpc-022a16d97e18fd465
- Owner:** 797666781058
- Inbound rules count:** 1 Permission entry
- Outbound rules count:** 1 Permission entry

Below the details, there are tabs for 'Inbound rules', 'Outbound rules', 'Sharing - new', 'VPC associations - new', and 'Tags'. The 'Inbound rules' tab is active, showing a table with one rule:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Desc
-	sg-0f1705c8867bf1ee4	-	PostgreSQL	TCP	5432	sg-0137e41ed35a5be2...	-

The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates.

Security Groups successfully created and configured. These stateful, virtual firewalls enforce the principle of least privilege, strictly controlling traffic flow. Rules permit external access to the ALB, load balancer traffic to the ECS Tasks, and application traffic to the backend RDS database.

Managed Data Layer: Launching the PostgreSQL RDS

The screenshot displays the AWS Management Console interface for a PostgreSQL RDS instance named **db-metabase**. The top navigation bar shows the AWS logo, a search bar with 'ECS', and the region 'United States (N. Virginia)'. The breadcrumb trail indicates the path: **Aurora and RDS** > **Databases** > **db-metabase**.

The instance details are presented in a summary card with the following information:

DB identifier	Status	Role	Engine	Recommendations
db-metabase	Available	Instance	PostgreSQL	
CPU	Class	Current activity	Region & AZ	
-	db.t3.micro		us-east-1b	

Below the summary card, a horizontal tab bar allows navigation between different sections: **Connectivity & security** (selected), **Monitoring**, **Logs & events**, **Configuration**, **Zero-ETL integrations**, **Maintenance & backups**, **Data migrations - new**, **Tags**, and **Records**.

The **Connectivity & security** section is expanded, showing three sub-sections:

- Endpoint & port:** The endpoint is `db-metabase.cqzg0yssggd9.us-east-1.rds.amazonaws.com` and the port is `5432`.
- Networking:** The instance is in the `us-east-1b` Availability Zone, connected to the `Metabase VPC (vpc-0dfb5ebba4ea6d2b4)` VPC and the `default-vpc-0dfb5ebba4ea6d2b4` Subnet group. The subnets listed are `subnet-0465476090d89acd9`, `subnet-05cfeecde4e2aac58`, and `subnet-02f2d0fc6a3a529e9`.
- Security:** The instance is associated with the `Metabase-SG (sg-0a56c27ee11701580)` VPC security group, which is `Active`. It is not publicly accessible. The certificate authority is `rds-ca-rsa2048-g1`, with a certificate authority date of `May 26, 2061, 06:34 (UTC+07:00)` and a DB instance certificate expiration date of `September 21, 2026, 16:04 (UTC+07:00)`.

The footer of the console shows the **CloudShell** icon, a **Feedback** link, and the copyright notice: © 2025, Amazon Web Services, Inc. or its affiliates. It also includes links for **Privacy**, **Terms**, and **Cookie preferences**.

A PostgreSQL RDS instance metabase-db was launched in private subnets. The endpoint and credentials were recorded.

Creating the ECS Fargate Cluster

On the Create Cluster page in the ECS console, the cluster type Networking only (Fargate) was selected and a cluster named metabase-ecs-cluster was created with the default VPC and networking configuration.

The screenshot shows the 'Create cluster' page in the Amazon ECS console. The page title is 'Create cluster' with an 'info' link. Below the title is a descriptive sentence: 'An Amazon ECS cluster groups together tasks, and services, and allows for shared capacity and common configurations. All of your tasks, services, and capacity must belong to a cluster.'

The main configuration section is titled 'Cluster configuration'. It contains a 'Cluster name' field with the value 'metabase-ecs-cluster'. Below the field is a note: 'Cluster name must be 1 to 255 characters. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).' There is a 'Service Connect defaults - optional' section with a right-pointing arrow.

The next section is 'Infrastructure - optional' with an 'info' link and a 'Serverless' button. It states: 'Your cluster is automatically configured for AWS Fargate (serverless) with two capacity providers. Add Amazon EC2 instances.' There are three options: 'AWS Fargate (serverless)' (selected with a blue checkmark), 'Amazon EC2 instances' (unselected), and 'External instances using ECS Anywhere' (unselected). The 'AWS Fargate (serverless)' option has a sub-note: 'Pay as you go. Use if you have tiny, batch, or burst workloads or for zero maintenance overhead. The cluster has Fargate and Fargate Spot capacity providers by default.'

Below this is the 'Monitoring - optional' section with a right-pointing arrow. It states: 'Configure observability, encryption, and logging options to maintain compliance and operational visibility of your container environment.'

Next is the 'Encryption - optional' section with a right-pointing arrow. It states: 'Choose the KMS keys used by tasks running in this cluster to encrypt your storage.'

The final section is 'Tags - optional' with an 'info' link and a right-pointing arrow. It states: 'Tags help you to identify and organize your clusters.'

At the bottom right of the form are two buttons: 'Cancel' and 'Create'.

The footer of the console shows 'CloudShell', 'Feedback', and copyright information: '© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences'.

Creating the ECS Fargate Cluster

The screenshot displays the AWS Management Console for the 'metabase-ecs-cluster'. The cluster is in an 'Active' state, as indicated by the green checkmark and the 'Active' status label. The 'Infrastructure' tab is selected, showing the capacity providers and container instances.

Cluster overview

- ARN: `arn:aws:ecs:us-east-1:797666781058:cluster/metabase-ecs-cluster`
- Status: **Active**
- CloudWatch monitoring: **Default**
- Registered container instances: **-**
- Services: **Draining** (Active)
- Tasks: **Pending** (Running)

Capacity providers (2)

Capacity provider	ASG	Man...	Man...	Man...	Curr...	Desi...	Min ...	Max ...	Update status	Update status re...
Fargate	-	-	-	-	-	-	-	-	-	-
Fargate Spot	-	-	-	-	-	-	-	-	-	-

Container instances (0)

Filter container instances by property or value

The Cluster Details page now highlights the cluster in an Active state, showing that it's fully set up and ready to deploy Fargate services without any further configuration needed.

Defining the Task: Container Specs and Settings

On the Create Task Definition page, a new Fargate task definition named metabase-task was set up with 1 vCPU, 3GB memory, and a container using the metabase/metabase:latest image mapped to port 3000. Environment variables for the PostgreSQL database connection were also entered.

The screenshot shows the 'Create new task definition' page in the AWS Management Console. The breadcrumb trail indicates the path: Amazon Elastic Container Service > Create new task definition. The page is titled 'Create new task definition' with an 'info' link. It is divided into several sections: 'Task definition configuration', 'Infrastructure requirements', 'Task roles - conditional', and 'Task execution role'. In the 'Task definition configuration' section, the 'Task definition family' is set to 'metabase-task'. The 'Infrastructure requirements' section shows 'Launch type' set to 'AWS Fargate'. Under 'OS, Architecture, Network mode', 'Operating system/Architecture' is 'Linux/X86_64' and 'Network mode' is 'awsvpc'. In the 'Task size' section, 'CPU' is set to '1 vCPU' and 'Memory' is set to '3 GB'. The 'Task roles - conditional' section shows a task role of '-'. The 'Task execution role' section shows 'ecsTaskExecutionRole'. At the bottom, there are expandable sections for 'Task placement - optional' and 'Fault injection - optional'.

Amazon Elastic Container Service > Create new task definition

Create new task definition [info](#)

Task definition configuration

Task definition family [info](#)
Specify a unique task definition family name.

metabase-task

Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.

▼ Infrastructure requirements
Specify the infrastructure requirements for the task definition.

Launch type [info](#)
Selection of the launch type will change task definition parameters.

☒ **AWS Fargate**
Serverless compute for containers.

☐ **Amazon EC2 instances**
Self-managed infrastructure using Amazon EC2 instances.

OS, Architecture, Network mode
Network mode is used for tasks and is dependent on the compute type selected.

Operating system/Architecture [info](#)
Linux/X86_64

Network mode [info](#)
awsvpc

Task size [info](#)
Specify the amount of CPU and memory to reserve for your task.

CPU
1 vCPU

Memory
3 GB

▼ Task roles - conditional

Task role [info](#)
A task IAM role allows containers in the task to make API requests to AWS services. You can create a task IAM role from the [IAM console](#).

-

Task execution role [info](#)
A task execution IAM role is used by the container agent to make AWS API requests on your behalf. If you don't already have a task execution IAM role created, we can create one for you.

ecsTaskExecutionRole

► Task placement - optional

► Fault injection - optional

Task Registered

The screenshot displays the AWS Elastic Container Service (ECS) console for a task definition named 'metabase-task:1'. The interface is in the 'United States (N. Virginia)' region. The breadcrumb navigation shows the path: Amazon Elastic Container Service > Task definitions > metabase-task > Revision 1 > Containers. The task definition is in an 'ACTIVE' state, last updated on September 21, 2025, at 16:23 (UTC+7:00). The overview section provides details such as the ARN, task role, fault injection status, time created, app environment, operating system, and network mode. The task size section shows the CPU and memory allocation for the metabase container and shared task resources.

metabase-task:1 Last updated September 21, 2025, 16:23 (UTC+7:00) Deploy Actions Create new revision

Overview [Info](#)

ARN arn:aws:ecs:us-east-1:797666781058:task-definition/metabase-task:1	Status ACTIVE	Time created September 21, 2025, 16:23 (UTC+7:00)	App environment EC2
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture Linux/X86_64	Network mode awsvpc
Fault injection Turned off			

Containers | JSON | Task placement | Volumes (0) | Requires attributes | Tags

Task size

Task CPU
2,048 units (2 vCPU)

Task CPU maximum allocation for containers

CPU (unit)

metabase	Shared task CPU
----------	-----------------

Task memory
4,096 MiB (4 GB)

Task memory maximum allocation for container memory reservation

Memory (MiB)

metabase	Shared task memory
----------	--------------------

The task definition was successfully registered and is now available as an active revision, ready for deployment of the Metabase container.

Creating the Service

On the Create Service page, an ECS service named metabase-service was configured to run one task in the private subnets. The ECS task security group and target group for traffic forwarding were also attached.

The screenshot shows the 'Create service' page in the AWS Management Console. The breadcrumb trail is 'Amazon Elastic Container Service > Task definitions > metabase-task > Create service'. The page is titled 'Create service' with an 'info' link. It is divided into two main sections: 'Service details' and 'Environment'.

Service details

- Task definition family:** A dropdown menu showing 'metabase-task' with a refresh icon.
- Task definition revision:** A text input field with a placeholder 'Enter a task definition revision' and a refresh icon.
- Service name:** A text input field containing 'metabase-task-service'. Below it, a note states: 'Up to 255 letters (uppercase and lowercase), numbers, underscores, and hyphens are allowed. Service names must be unique within a cluster.'

Environment

Existing cluster: A dropdown menu showing 'metabase-ecs-cluster' with a refresh icon and a 'Create a new cluster' button.

Compute configuration - advanced

Compute options

To ensure task distribution across your compute types, use appropriate compute options.

- Capacity provider strategy:** A radio button is selected. The description says: 'Specify a launch strategy to distribute your tasks across one or more capacity providers.'
- Launch type:** A radio button is unselected. The description says: 'Launch tasks directly without the use of a capacity provider strategy.'

Capacity provider strategy

Select either your cluster default capacity provider strategy or select the custom option to configure a different strategy.

- Use cluster default:** An unselected radio button. The description says: 'No default capacity provider strategy configured for this cluster.'
- Use custom (Advanced):** A selected radio button.

Capacity provider

A dropdown menu showing 'FARGATE' with a refresh icon.

Base

A text input field containing '0'.

Weight

A text input field containing '1'.

Add capacity provider

A button with a plus icon.

Platform version

A dropdown menu showing 'LATEST' with a refresh icon.

Troubleshooting configuration - recommended, new

A link with a plus icon.

Service Running

The screenshot shows the AWS Management Console interface for the 'metabase-ecs-cluster'. The top navigation bar includes the AWS logo, a search bar, and the region 'United States (N. Virginia)'. The breadcrumb trail indicates the path: Amazon Elastic Container Service > Clusters > metabase-ecs-cluster > Services.

metabase-ecs-cluster

Last updated: September 21, 2025, 16:57 (UTC+7:00)

Buttons: Update cluster, Delete cluster, Launch

Cluster overview

ARN arn:aws:ecs:us-east-1:797666781058:cluster/metabase-ecs-cluster	Status Active	CloudWatch monitoring Default	Registered container instances -
---	-------------------------	---	--

Services

Draining -	Active 1	Pending -	Running 1
----------------------	--------------------	---------------------	---------------------

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Tags

Services (1)

Last updated: September 21, 2025, 16:57 (UTC+7:00)

Buttons: Manage tags, Update, Delete service, Create

Filter services by value

Filter launch type: Any launch type

Filter scheduling strategy: Any scheduling strategy

☐	Service name	ARN	Status	Schedu...	Lau...	Task de...	Deployments and tasks	Last deployment
☐	metabase-task-service	arn:aws:ecs:us-east-1:797666781058:cluster/metabase-ecs-cluster/service/metabase-task-service	Active	REPLICA	-	metabase...	1/1 Tasks running	Completed View

The Service Details page shows the service in a Running and Stable state, with the status marked as Active, confirming that traffic is now seamlessly routed to the ECS tasks through the load balancer.

Accessing the Service via Public IP

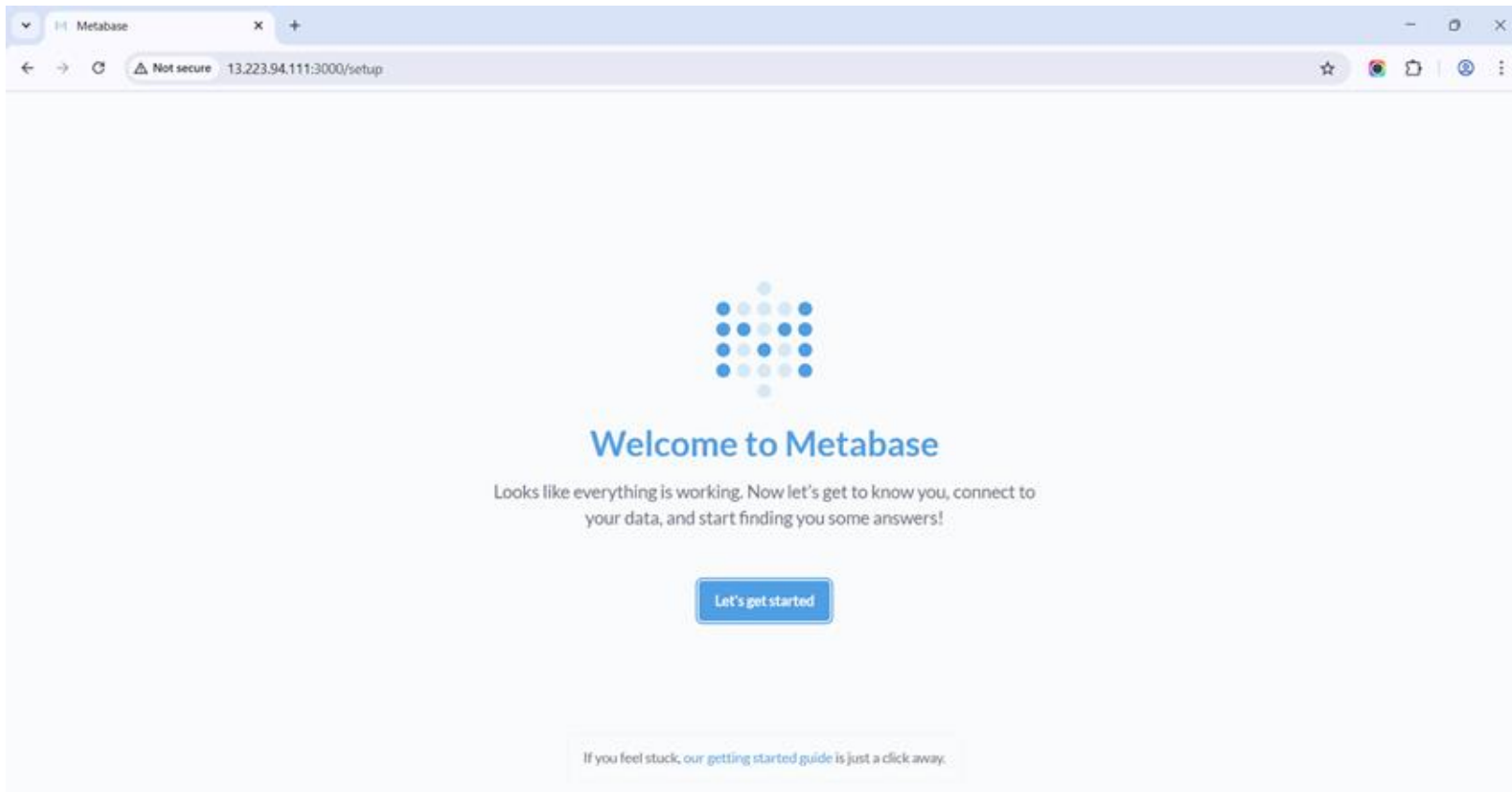
The screenshot displays the AWS Management Console interface for configuring an Amazon Elastic Container Service (ECS) Fargate task. The breadcrumb navigation at the top indicates the path: Amazon Elastic Container Service > Clusters > metabase-ecs-cluster > Tasks > 194d9b1a61584a9a92ffaf8a89c7cc70 > Configuration. The main content area is titled 'Configuration' and is organized into several sections:

- Operating system/Architecture:** Linux/X86_64
- CPU | Memory:** 1 vCPU | 3 GB
- Platform version:** 1.4.0
- Task execution role:** [ecsTaskExecutionRole](#)
- Task role:** -
- Fault injection:** Turned off
- ECS Exec:** Turned off
- Task scale-in protection:** Protection status is Turned off, and Protection duration is -.
- Capacity provider:** Fargate
- Launch type:** Fargate
- Task definition: revision:** [metabase-task:3](#)
- Task group:** service:metabase-task-service | [View service](#)
- ENI ID:** [eni-0b1589ea49c69bdd9](#)
- Network mode:** awsvpc
- Subnet:** [subnet-0465476090d89acd9](#)
- Public IP:** [13.223.94.111](#) | [open address](#)
- Private IP:** [10.0.0.142](#)
- MAC address:** [02:10:ff:8e:72:79](#)

Below the configuration section, there is a 'Containers (1)' section with a search bar labeled 'Filter containers'. At the bottom of the console, the 'Container details for metabase' section is partially visible. The footer of the console shows the CloudShell icon, a Feedback link, and copyright information for Amazon Web Services, Inc. or its affiliates, along with links for Privacy, Terms, and Cookie preferences.

Confirmation of the successful launch of the ECS Fargate task, showing key details such as task ID, public and private IP addresses (if assigned).

Seeing It in Action: Accessing Metabase



Open this URL in a web browser to confirm the application is accessible and running correctly on the deployed Metabase.

Database Check: Verifying PostgreSQL Connectivity

The screenshot displays the Metabase Admin interface for a PostgreSQL database. The top navigation bar includes links for Metabase Admin, Settings, Databases, Table Metadata, People, Permissions, Performance, Tools, and an Exit admin button. The main content area is titled 'Databases > POSTGRESQL' and 'PostgreSQL', with a subtitle 'PostgreSQL Added September 21, 2025'. The 'Connection and sync' section shows the database is 'Connected' and provides buttons for 'Sync database schema' and 'Re-scan field values'. The 'Model features' section includes a toggle for 'Model actions' and a note about permissions. The 'Danger zone' at the bottom contains buttons for 'Discard saved field values' and 'Remove this database'.

Metabase Admin Settings **Databases** Table Metadata People Permissions Performance Tools [Exit admin](#)

DATABASES > POSTGRESQL

PostgreSQL

PostgreSQL Added September 21, 2025

Connection and sync

Manage details about the database connection and when Metabase ingests new data.

Connected [Edit connection details](#)

[Sync database schema](#) [Re-scan field values](#)

Model features

Choose whether to enable features related to Metabase models. These will often require a write connection.

Model actions

☐ Allow actions from models created from this data to be run. Actions are able to read, write, and possibly delete data. Note: Your database user will need write permissions.

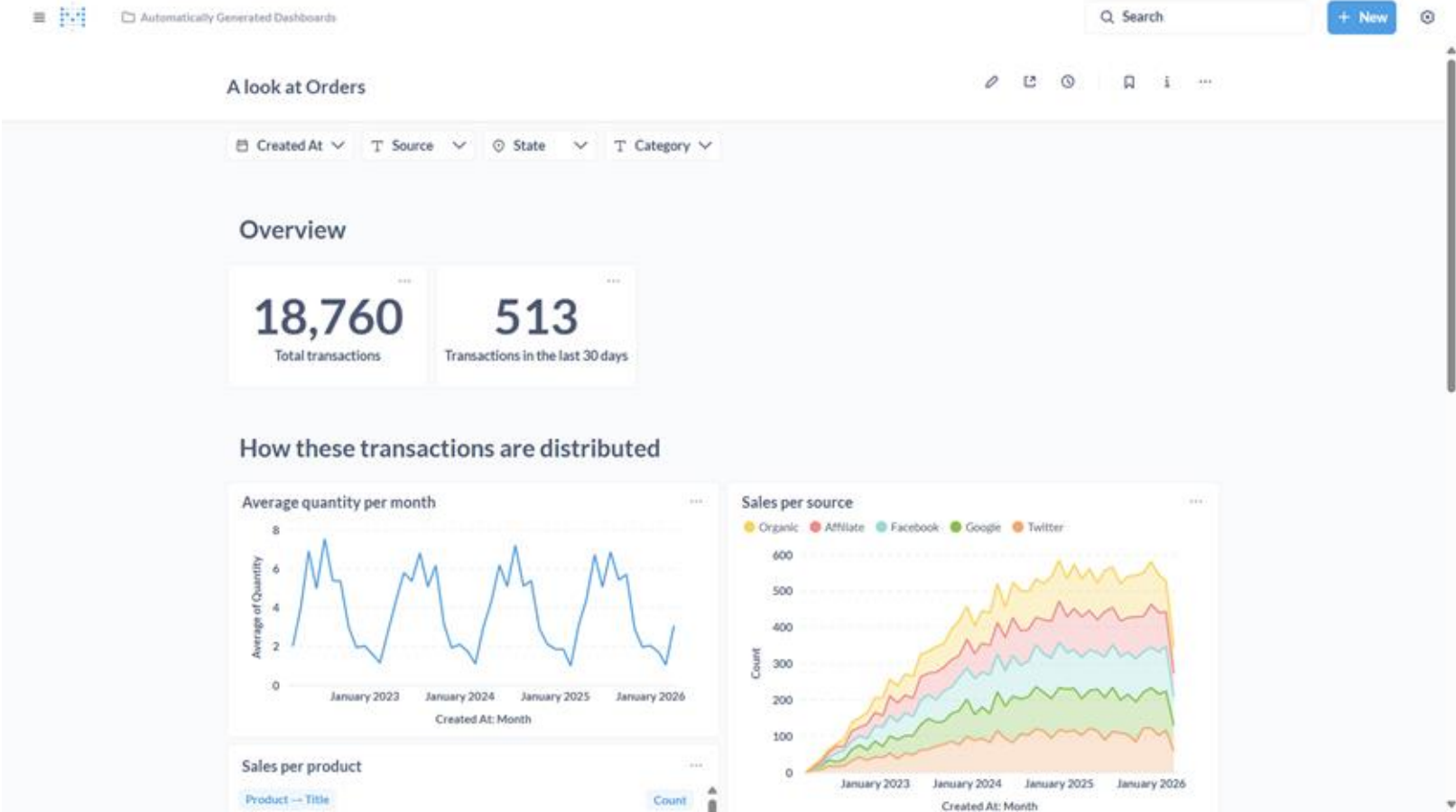
Danger zone

Remove this database and other

[Discard saved field values](#) [Remove this database](#)

Confirm that Metabase successfully connects to the PostgreSQL RDS

Sample Metabase Dashboard



Best Practices for Production Deployment

- For a production environment, it is strongly recommended to use an **Application Load Balancer (ALB)** to ensure high availability, automatic traffic distribution, and easier scaling of ECS tasks.
- ALB provides built-in health checks and better traffic routing, which improves reliability and performance compared to exposing ECS tasks directly.
- Although this setup was tested on a **free tier account** where ALB is **not fully supported**, following the ALB-based architecture remains the industry best practice for secure and scalable deployments.
- If free tier limitations are a concern, consider upgrading the AWS account or using a paid plan to leverage ALB features for long-term stability.

Lessons from the Deployment Journey

- A complete Metabase environment was successfully deployed on AWS using **ECS Fargate** and **PostgreSQL RDS**, with a network architecture designed for both scalability and security.
- The environment follows a production-ready design, separating public and private subnets and implementing security groups for controlled access.
- While the current free tier environment cannot fully implement ALB, the architecture has been built with ALB integration in mind, ensuring an easy upgrade path to a production-ready setup when moving to a paid plan.
- This approach provides a solid foundation to scale, secure, and manage Metabase in a cloud-native infrastructure.

Thank You